# Chapter 19: Data Structures for Disjoint Sets

## Disjoint Set Operations

A **disjoint set** is a data structure that keeps track of collection of disjoint dynamic sets. Each **set** has a **representative** element that serves as an identifier for the set. We want to perform the following operations on disjoint sets:

MAKE-SET$(x)$ creates a new set containing the single element $x$. The representative of the set is $x$ itself.

UNION$(x, y)$ unites the sets that contain $x$ and $y$, say $S_x$ and $S_y$, into a single set that is the union of these two sets. The representative of the union is either the representative of $S_x$ or the representative of $S_y$.

FIND-SET$(x)$ returns a pointer to the representative of the set containing x. If x is a representative, then FIND-SET(x) returns x.

We are interested in performing a sequence of $m$ MAKE-SET, UNION, and FIND-SET operations collection of $n$ elements as efficiently as possible.

In the begining, we have $n$ MAKE-SET operations, which implies that $m \geq n$.

Each UNION operation reduces the number of sets by one, so there can be at most $n-1$ UNION operations.

## Linked-List Representation of Disjoint Sets

Each set is represented by a linked list. The representative of the set is the first node in the linked list. Each node in the linked list contains a pointer to the next node, a pointer to the representative of the set (the first node), and a pointer to the tail of the list.

This allows us to perform MAKE-SET and FIND-SET in $O(1)$ time. However, the UNION operation takes $O(n)$ time, since we have to update the representative pointer of each node in the smaller list to point to the representative of the larger list. Therefore, the total time for $m$ operations is $O(m+n^2)$.

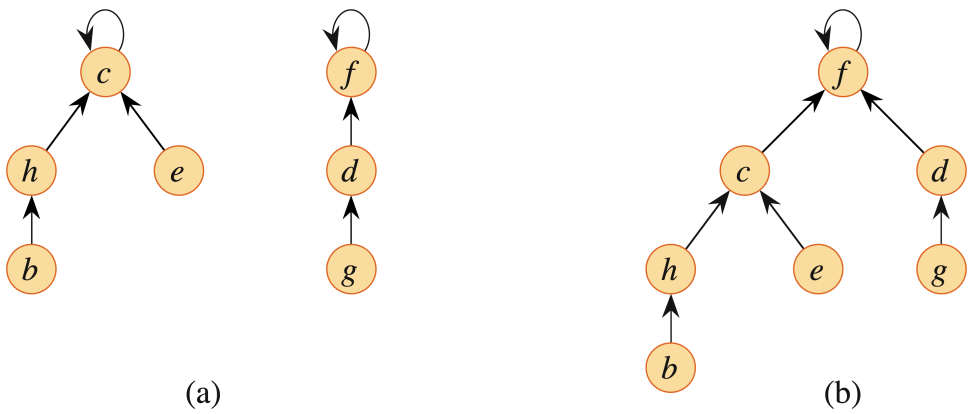
**weighted union heuristic**: always append the smaller list to the larger list. This reduces the time for UNION to $O(\min\{|S_x|, |S_y|\})$. The total time for $m$ operations is $O(m+n\log n)$.

# Disjoint Set Forests

A faster way to represent disjoint sets is to use a **disjoint set forest**, which is a collection of rooted trees. Each tree represents a set, and the root of the tree is the representative of the set. Each node in the tree contains a pointer to its parent, and the root node has a pointer to itself.

The figure (part-a) below shows two tress in a disjoint set forest. Each tree represents a set, and the root of the tree is the representative of the set. Each node in the tree contains a pointer to its parent, and the root node has a pointer to itself.

The figure (part-b) below shows the result of performing a UNION operation on the two sets represented by the two trees in part-a. The tree on the left is appended to the tree on the right, and the root of the left tree becomes a child of the root of the right tree.




This allows us to perform MAKE-SET in $O(1)$ time, since we can create a new tree with a single node. The UNION operation can be performed in $O(1)$ time by making the root of one tree point to the root of the other tree. However,FIND-SET can take $O(n)$ time in the worst case, since we may have to traverse a long path from a node to the root.

- **union by rank heuristic**: always make the root of the smaller tree point to the root of the larger tree. This reduces the time for FIND-SET to $O(\log n)$, since the height of the trees is at most $\log n$. The total time for $m$ operations is $O(m + n \log n)$.
  - To implement this heuristic, we maintain an additional array *rank* that stores the rank of each node, which is an upper bound on the height of the subtree rooted at that node. When we perform a UNION operation, we compare the ranks of the roots of the two trees and make the root with smaller rank point to the root with larger rank. If the ranks are equal, we can choose either root to be the new root and increment its rank by one.
- **path compression heuristic**: during a FIND-SET operation, make each node on the path from the node to the root point directly to the root. This reduces the time for FIND-SET to $O(\alpha(n))$, where $\alpha(n)$ is the inverse Ackermann function, which grows very slowly. The total time for $m$ operations is $O(m\alpha(n))$.

```
MAKE-SET(x)
1.  x.p = x
2.  x.rank = 0

UNION(x, y)
1.  Link(FIND-SET(x), FIND-SET(y))o

Link(x, y)
1.  if x.rank > y.rank
2.      y.p = x
3.  else
4.      x.p = y
5.      if x.rank == y.rank
6.          y.rank = y.rank + 1

FIND-SET(x)
1.  if x != x.p
2.      x.p = FIND-SET(x.p)
3.  return x.p
```

See code example here: disjoint sets example.